WimPyDD Documentation

Sunghyun Kang, Stefano Scopel, Gaurav Tomar

Department of Physics, Sogang University, Seoul, Korea, 121-742

To get more details and follow updates, please visit our homepage:
wimpydd.hepforge.org

Classes-

- `element(symbol,verbose=False)`

Initializes an element of the periodic table
A set of elements is pre–defined in WimPyDD. Type `list_elements()` to list them.

Parameters:

`symbol(str)` - A string starting with capital letter with the symbol of the element. Must match the name of the file symbol+'.tab' in the directory WimPyDD/Targets. By default uses the nuclear form factor calculated by Anand et al, Phys. Rev.C 89, 065501 (2014) or Catena et al., JCAP04(2015)042. A custom nuclear form factor can be used if the file isotope_name+'_func_w.py' is present in WimPyDD/Targets/Nuclear_response_functions/. isotope_name must be one of the strings contained in the isotopes attribute array.
Example: Xe=`WimPyDD.element('Xe')` requires the file WimPyDD/Targets/Xe.tab and, if a custom form factor is used for ^{133}Xe, the file WimPyDD/Targets/
Nuclear_response_functions/133xe_func_w.py. The array `Xe.isotopes` must contain the string '133Xe'
`verbose(bool)`- if True prints out details of accessed external files. Default=False

Attributes:

`a(array)` - array containing the atomic mass numbers of the isotopes of the element
`abundance(array)` - array containing the natural fractional abundance of the element isotopes
`average_a(float)` - element atomic mass number averaged over isotopes
`average_mass(float)` - element mass in GeV averaged over isotopes
`isotopes(array)` - array of strings with the symbols of the isotopes of the element
`itar(array)` - array containing internal codes to access the default Nuclear functions coefficients in WimpyDD/Targets/Nuclear_response_functions/nuclear_response _functions_coefficients_table.dat
`mass(array)` - array containing the atomic masses in GeV of the isotopes of the element
`n_isotopes(int)` - the number of isotopes

name(str) - name in full of the element
nt_kg(array) - array containing the number of targets per kg of the isotopes of the element (for a sample of pure element)
nt_kg_average(float) - number of targets per kg averaged over the isotopes of the target
spin(array) - array containing the spins of the isotopes of the element (in multiples of 0.5)
symbol(str) - symbol of the element
z(int) - atomic number of the element
func_w(array) - array of functions w(q) containing the nuclear form factors of the isotopes of the element,

w(q) - element of the array func_w
This function defined locally calculates the 8 nuclear form factors,
$M, \Sigma', \Sigma'', \Phi'', \widetilde{\Phi}', \Delta, \Phi'' M, \Delta\Sigma'$ of the isotopes
Input:
q - exchanged momentum in GeV
Output:
array with shape (8,2,2) containing 8, 2×2 matrices, each for every form factor X=$M, \Sigma', \Sigma'', \Phi'', \widetilde{\Phi}', \Delta, \Phi'' M, \Delta\Sigma'$. Each 2x2 matrix represents W_X(tau,tau_prime) with tau=0, 1, tau_prime=0, 1 for two isospin indices.

- target(formula, verbose=False)

Initializes the target with given elements combining objects belonging to element class

Parameters:

formula(str) - a string starting with combination of elements. The first letter of each element should be capital letter. Each element is used as an object belonging to element class.
Example: C3F8=WimPyDD.tartet('C3F8') uses WimPyDD.element('C') and WimPyDD.element('F')
verbose(bool) - If True prints out details of accessed external files. Default=False.

Attributes:

a_tot(float) - a number of total atomic mass number
element(array) - an array containing the class elements of the formula
formula(str) - name of target formula
mass(float) - sum of total average mass of elements
n(array) - an array containing the number of each element
n_targets(int) - the number of element types
nt_kg(array) - an array containing total mass in kg unit per mol of each element

---

- `eft_hamiltonian(name, wilson_coefficients, q_norm=1.)`

Sets the hamiltonian based on effective field theory

Parameters:

`name(str)` - name of hamiltonian model
`wilson_coefficients(dict)` - a ditctionary containing couplings
`q_norm(float)` - normalization for q. Default=1
Example: spin_independent= `WimPyDD.eft_hamiltonian`('spin_independent',{0:lambda r:[1.+r,1.-r] }). In this way, we can name the model as spin_independent which is the name of folder made during calculation of response functions with this model

Attributes:

`arguments(dict)` - a dictionary containing given Wilson coefficients as key and arguments as value
`coeff_squared_list(list)` - a list containing possible squared coefficients
`couplings(list)` - a list containing given Wilson coefficients
`global_arguments(array)` - an array containing all arguments of Wilson coefficients
`global_default_args(dict)` - a dictionary containing default arguments to track global arguments correctly
`hamiltonian(str)` - hamiltonian in form of coupling times operator
`name(str)` - given name parameter
`q_norm(float)` - given normalization for q in order to extract momentum dependence from Wilson coefficients to treat it in different way
`wilson_coefficients(dict)` - a dictionary containing given Wilson coefficients as key and given functions as value
`wilson_coefficients_list(list)` - a list containing given Wilson coefficients

Methods:

`coeff_squared(c1,c2,flatten=False,q_norm=q_norm,**args)`:
Input:
`c1` - Wilson coefficient among given parameter
`c2` - Wilson coefficient among given parameter
`flatten` - if True, returns 2×2 matrix of all value as 1. Default=1
`q_norm` - normalization for q. Default is same as given value
Output:
Returns Kronecker product of two given Wilson coefficients

`coeff_squared_q_dependence(q,c1,c2,q_norm=q_norm)`:
Input:
`q` - momentum
`c1` - Wilson coefficient among given parameter
`c2` - Wilson coefficient among given parameter
`q_norm` - normalization for q. Default is same as given value
Output:

Returns dependence of q

`print_hamiltonian(self):`
Input:
self
`>>>WimPyDD.eft_hamiltonian.print_hamiltonian()`
Output:
Returns combination of coupling times operator

`print_hamiltonian_latex(self):`
Input:
self
`>>>WimPyDD.eft_hamiltonian.print_hamiltonian_latex()`
Output:
Returns combination of coupling times operator in latex form

- `experiment(name, target\textunderscore name, exposure=1., verbose=False)`

Creates experiment and its folder inside './Experiments' unless there is already existing one. It reads efficiency, modifier, quenching, resolution and data inside corresponding directory. If not given, they are automatically set to be 1

Parameters:

`name(str)` - name of experiment
`target_name(str)` - formula used in class target
`exposure(float)` - exposure of experiment. Default=1
`verbose(bool)` - If True prints out details of accessed external files. Default=False
Example: test_experiment=`WimPyDD.experiment('test_experiment','C3F8',365.)`

Attributes:

`exposure(float)` - given experiment exposure in kg-day unit.
`name(str)` - given experiment name.
`response_functions(tuple)` - a tuple containing calculated response functions.
`target` - given experiment target object belonging to class target.
`data(documented_tuple)` - a tuple with information documents containing arrays of data

Methods:

`data(self):`
Input:
`self`

`>>>WimPyDD.experiment.data()`
Output:
Returns tuple containing experimental data. If not given, automatically set to be 1

`efficiency(e):`
Input:
`e` - energy.
Output:
Returns efficiency at given energy. If not given, automatically set to be 1

`modifier(er,eprime,exp,n_element,n_isotope,n_bin):`
Input:
`er(float)` - recoil energy
`eprime(float)` - visible energy
`n_element(int)` - the number of elements
`n_isotope(int)` - the number of isotopes
`n_bin(int)` - the number of energy bin
Output:
Function which used to modify response functions in correct way. If not given, automatically set to be 1

`resolution(args):`
Input:
`visible energy(float)` - eprime for ionizators and scintillators, photo electron for xenon detectors
`recoil energy(float)` - electro equivalent energy for ionizators and scintillators, recoil energy for xenon detectors.
Output:
Returns energy resolution at given arguments. If not given, automatically set to be 1

Functions-

All the functions provided with the `WimPyDD` are briefly described as following,

- `wimp_dd_rate(exp,hamiltonian,vmin,delta_eta,mchi,delta)`

Calculates the number of events predicted in each energy bin defined in experiment `exp`

Parameters:

`exp` - object belonging to `experiment` class
`hamiltonian` - object belonging to `eft_hamiltonian` class
`vmin` - array containing a list of WIMP stream speed velocities in the lab frame in km/s.
`delta_eta` - array containing the contribution of each stream to the halo function $\eta(v)$ in $(\mathrm{km/sec})^{-1}$

mchi - WIMP mass in GeV
delta - mass splitting for inelastic scattering, in keV. Default=0
j_chi - WIMP spin. Default=0.5
rho_loc - local dark matter density in the unit of GeV/cm^3. Default=0.3
velocity_dependence - velocity depenence in the cross-section. Default=True
response_functions - input response function. Default=None
energy_bins_vec - this facilitate the user to calculate the number of events in particular bins. Default=None
flatten=False -*******
reset_response_functions=False -******
print_contributions - if True, the user could check the contribution of each Wilson coefficient separately. Default=None
**args - these are arguments which are defined by the user e.g. isospin-violating parameter r, momentum transfer q. By default their values are set to 1

Output:

An array containing the predicted number of events in each energy bin of defined experiment exp

---

- dsigma_der(element,hamiltonian,mchi,delta,j_chi,v,er)

Calculates the differential cross-section for WIMP-nucleus scattering

Parameters:

element - object belonging to element class
hamiltonian - object belonging to eft_hamiltonian class
mchi - WIMP mass in GeV
delta - mass splitting for inelastic scattering, in keV. Default=0
j_chi - WIMP spin
v - WIMP incoming velocity in km/s
er - nuclear recoil energy in keV
velocity_dependence - velocity dependence in the cross-section. Default=True
**args - these are arguments which are defined by the user e.g. isospin-violating parameter r, momentum transfer q. By default their values are set to 1

Ouput:

An array containing the differential cross section normalized to cm^2/keV for each isotopes (ni=element.n_isotopes) contained in the element. If v is a one-dimensional array of nv values the shape of the output is (ni,nv)

- load_response_functions(exp,hamiltonian)

Loads the response functions for defined experiment `exp`

Parameters:

`exp` - object belonging to `experiment` class
`eft_hamiltonian` - object belonging to `eft_hamiltonian` class
`j_chi` - WIMP spin. Default=0.5
`reset` - If True, removes previously existing response functions
`verbose(bool)` - If True prints out details of accessed external files. Default=True

Output:

A dictionary of response functions which could be accessed by
`exp.response_functions[hamiltonian,j_chi]`

- `eft_amplitude_squared(coeff1,q,element,isotope)`

Calculates WIMP-nucleus scattering square amplitude

Parameters:

`coeff1` - either a (18,2) dimensional array containing the couplings in units of GeV^{-2} of the non-relativistic effective theory c_n^τ = `coeff1` $(n-1,\tau), n = 1-18, \tau = 0,1$ in the notations of Anand et al., PHYSICAL REVIEW C89, 065501 (2014) and Dent et al., Phys.Rev. D92(2015)no.6, 063515 Example: c_4^1 = `coeff1`$(3,1), c_{11}^0$ =`coeff1`(10,0)
or a dictionary with c^τ = `coeff1`$([1-\tau,\tau])$ in our all spins notation Example: `coeff1`$=\{(`M', 0, 0) : [0,1]\}$
`q` - exchanged momentum in GeV
`element` - object belonging to `element` class
`isotopes(int)` - one of the isotopes of the `element` in integer form
`coeff2` - share similar structure like `coeff1`. If None, `coeff1=coeff2`. Default=None
`j_chi` - WIMP spin. Default=0.5

Output:

Squared amplitude in units of GeV^{-4}.

- `get_vstar(mn,mchi,delta)`

Calculates minimal WIMP incoming speed in the lab frame for inelastic scattering

Parameters:

`mn` - Nuclear mass in GeV
`mchi` - WIMP mass in GeV
`delta` - mass splitting for inelastic scattering, in keV

Output:

velocity in the units of km/s

- `get_estar(mn,mchi,delta)`

Calculates recoil energy corresponding to minimal WIMP incoming speed in the lab frame for inelastic scattering

Parameters:

`mn` - Nuclear mass in GeV
`mchi` - WIMP mass in GeV
`delta` - mass splitting for inelastic scattering, in keV

Output:

recoil energy in keV.

- `er_min(mn,mchi,delta,vmin)`

Calculates the minimum value of recoil energy for a target nuclei

Parameters:

`mn` - Nuclear mass in GeV
`mchi` - WIMP mass in GeV
`delta` - mass splitting for inelastic scattering, in keV
`vmin` - minimum WIMP velocity (in lab frame) required to deposit a recoil energy in km/s

Output:

minimum recoil energy in keV

- `er_max(mn,mchi,delta,vmin)`

Calculates the maximum value of recoil energy for a target nuclei

Parameters:

`mn` - Nuclear mass in GeV
`mchi` - WIMP mass in GeV
`delta` - mass splitting for inelastic scattering, in keV
`vmin` - minimum WIMP velocity (in lab frame) required to deposit a recoil energy in km/s

Output:

maximum recoil energy in keV

- `get_vmin(er,mn,mchi,delta)`

Calculates minimal velocity in lab frame which is required to deposit a given recoil energy in the detector

Parameters:

`er` - nuclear recoil energy in keV
`mn` - Nuclear mass in GeV
`mchi` - WIMP mass in GeV
`delta` - mass splitting for inelastic scattering, in keV

Output:

minimum velocity in the units of km/s

- `diff_rate(target,hamiltonian,mchi,spin,er)`

Calculates the differential rate for WIMP-nucleus scattering

Parameters:

`target` - object belonging to `target` class
`hamiltonian` - object belonging to `eft_hamiltonian` class
`mchi` - WIMP mass in GeV
`j_chi` - WIMP spin
`er` - nuclear recoil energy in keV
`vmin` - array with values of vmin in km/sec where delta_eta is sampled
`delta_eta` - array containing the contributions of streames vmin to the halo function in $(\mathrm{km/sec})^{-1}$
`delta` - mass splitting for inelastic scattering, in keV. Default=0
`exposure` - exposure of experiment. Default=1
`rho` - local dark matter density in $\mathrm{GeV/cm^3}$. Default=0.3

**args - these are arguments which are defined by the user e.g. isospin-violating parameter r, momentum transfer q. By default their values are set to 1.

Output:

differential rate in units of events/kg/day/keV

- `response_functions_list(exp)`

Lists response functions for defined experiment `exp`

Parameters:

`exp` - object belonging to `experiment` class

Output:

Information about the loaded response functions for defined experiment `exp`.

- `plot_response_functions(exp,eft_hamiltonian,j_chi)`

Plots response functions for defined experiment `exp`

Parameters:

`exp` - object belonging to `experiment` class
`eft_hamiltonian` - object belonging to `eft_hamiltonian` class
`j_chi` - WIMP spin
`coeff_squared_list` - if provided, consider only supplemented coefficients sqaured list. Example: [(('Omega', 0, 0), ('Omega', 0, 0))]; (default=None)
`filename_suffix` - **********
`tuple` - if provided, consider only supplemented response functions tuple (default=None)
`entry_card` - if no entry card is provided plots all the response functions contained in `exp.response_functions`. Otherwise plots only response function correspondingly
Example: `entry_card=(n_vel,tau,tau_prime,n_bin,n_element,n_isotope)`
`rescaling_factor` - factor to rescale response functions. Default=1
`flatten` - if True, load flatten response functions Default=False

Output:

Figure showing response functions for defined experiment `exp`

- `streamed_halo_function_maxwellian(v_rot_gal=[0.,220.,0.], v_sun_rot=[9.,12.,7.])`

Calculates `eta/delta_eta` halo function for maxwellian velocity distribution

Parameters:

`v_rot_gal` - galactic rotation velocity. Default=220 km/s
`v_sun_rot` - proper motion of sun in galactic coordinates
`v_esc_gal` - galactic escape velocity. Default=550 km/s
`n_vmin_bin` - number of streams. Default=50
`yearly_modulation` - if True, includes modulation calculating delta_eta1. Default=False)
`vrms` - root-mean-square velocity. Default=None
`vmin` - an array of vmin. Default=None
`v_earth_sun` - earth orbital velocity around sun
`delta_eta1`. **********. If False calculates eta halo function. Default=True

Output:

A tuple with arrays of vmin, `eta/delta_eta` with shape of `n_vmin_bin` or `vmin.shape`

- `streamed_halo_function(velocity_distribution_gal=None)`

calculates `eta/delta_eta` halo function for provided velocity distribution. There exist 8 possibilities for the calculation of `eta/delta_eta` using analytical (power expansion) or numerical methods. The eight possibilities include: velocity distribution (2: None or provided by user)-power expansion (2: True or False)-yearly_modulation (2: True or False)

Parameters:

`velocity_distribution_gal` - input velocity distribution in the galactic frame. If not provided, a maxwellian velocity distribution is considered in the galactic frame. Default=None
`v_rot_gal` - galactic rotation velocity. Default=220 km/s
`v_sun_rot` - proper motion of sun in the galactic coordinates
`v_esc_gal` - galactic escape velocity. Default=550 km/s
`n_vmin_bin` - number of velocity streams. Default=50
`yearly_modulation` - if True, includes modulation calculating `delta_eta1`. Default=False
`vmin` - an array of vmin. Default=None
`delta_eta1`. **********. If False calculates eta halo function. Default=True
`power_expansion` - if True, calculates `delta_eta` using power expansion while opposite performs numerical calculation. Default=False
`outputfile` - if string provided, perform the numerical calculation and stores the output tuple with given name. If file already exists, directly load the file. Default=None
`recalculate` - if True, recalculates the output tuple. Default=False
`modulation_phase` - modulation phase. Default=2nd June or 152.5 days
`**args` - any other parameters of velocity distribution. Example: root-means-square velocity (vrms),

most probable WIMP speed (v0)

Output:

A tuple of arrays of vmin, `eta/delta_eta` with shape of `n_vmin_bin` or `vmin.shape`

- `mchi_sigma_exclusion(exp, hamiltonian, vmin, delta_eta, delta=0.)`

Calculates the 90%C.L. upper bound of the conventional WIMP-nucleon cross section $\sigma_p = c^2\mu^2/\pi$ (in cm^2) as a function of the WIMP mass (in GeV) for a the halo function contained in delta_eta

Parameters:

`exp` - object belongs to experiment class
`hamiltonian` - object belongs to hamiltonian class
`vmin` - array containing a list of WIMP stream speed velocities in the lab frame in km/s
`delta_eta` - array containing the contribution of each stream to the halo function eta(v) in $(km/sec)^{-1}$
`delta` - mass splitting for inelastic scattering in keV. Default=0
`mchi_range` - WIMP mass range in GeV
`mchi_scale` - if linear, a linear scale is used for mchi vec. Default='linear'
`n_points` - number of points in mchi vec. Default=1000
`rho_loc` - local WIMP density in GeV/cm^3
`velocity_dependence` - velocity dependence in the cross-section. Default=True
`data_points_range` - experiment bins to be included. Example- [0] for first bin. Default=None
`elements_vec` - element to be included from the target of experiment. Example- [0] or [1] for DAMA targets NaI. Default=None
`j_chi` - WIMP spin. Default=0.5
`response_functions` - desired response functions can be passed through directly. Default=None
`energy_bins_vec` - how it is different from data-points-range
`flatten` - if True, load flatten response functions. Default=False
`**args` - these are arguments which are defined by the user e.g. isospin-violating parameter r, momentum transfer q. By default their values are set to 1

Output:

A tuple containing arrays of WIMP mass (GeV) and cross-section (cm^2)

- `v_earth_sun(ecliptic_longitude,verbose=False,v_rot_gal=[0.,220.,0.])`

Calculates earth orbital velocity relative to sun

Parameters:

`ecliptic_longitude` - ecliptic_longitude
`verbose` - if True prints the information
`v_rot_gal` - galactic rotation velocity. Default=220 km/s
`v_sun_rot` - proper motion of sun in galactic coordinates

Output:

An array containing earth orbital velocity relative to sun
